

System Architecture & Folder Directory Guide

Sovereign Digital Public Infrastructure (DPI) Entry Point for Citizen Participatory Budgeting and Municipal CapEx Planning across BRICS+ Nations

1. Executive Summary & Core Platform Overview

CivicPulse-BRICS is a Sovereign Digital Public Infrastructure (DPI) web application built to connect citizens, municipal city engineers, and central government leadership across 11 BRICS+ member states. The application integrates real-time citizen complaint tracking, AI-powered municipal MegaPlan synthesis (via Google Gemini), multilingual GNMT translation across 20+ regional languages, Leaflet GIS command mapping, and cross-border strategic CapEx project upvoting.

2. High-Level System Architecture & Flowchart

Layer / Module	Components & Responsibilities
Frontend (Browser)	HTML5 / Vanilla JS / Enterprise CSS Design System. • login.html: Sovereign Login & Authentication Gateway • citizen.html: Citizen Portal (Issue Reporting, GPS, AI Voice, Demands) • city_official.html: Municipal Operations & Resolution Dispatch • government.html: Central Government Hub & BRICS CapEx Analytics • central_official.html: Strategic BRICS Incoming Request Portal
Backend Gateway	backend/server.py (FastAPI / Starlette ASGI Engine) Central router handling authentication, API endpoints (/api/complaints, /api/demands, /api/brics, /api/auth), static asset delivery, and file uploads.
Data & Storage	• data/civimpulse.db (Primary SQLite DB): Complaints, Users, SLA records, SMS logs • data/demands.json: Community Demand proposals and voter tracking • data/city_proposals.json: Municipal CapEx proposals submitted to Centre • data/brics_incoming_requests.json: Cross-border BRICS Joint Ventures
AI & Microservices	• gemini_helper.py: Google Gemini AI Triage & Plan Generation • megaplan_engine.py: Automated Municipal MegaPlan Synthesis • brics_context.py: Geopolitical metadata for 11 BRICS+ nations • messaging_gateway.py & sms_gateway.py: SMS/WhatsApp alerts • public_data_helper.py: Live Weather & Environmental context

3. End-to-End Request Lifecycle Flowcharts

Flow A: Citizen Complaint Submission

Citizen fills complaint & pins GPS location -> **citizen.js** calls POST /api/complaints -> **server.py** validates photo -> **gemini_helper.py** performs AI severity triage -> **api_client.py** writes to *data/civimpulse.db* -> **messaging_gateway.py** sends SMS/WhatsApp notification to citizen.

Flow B: GPS Reverse Geocoding

User clicks 'Pin on Map' -> **citizen.js** captures lat/lon via Leaflet -> calls GET /api/location/reverse -> **server.py** queries OpenStreetMap Nominatim -> returns street address & BRICS ward to form.

Flow C: Municipal Complaint Resolution

City Engineer opens **city_official.html** -> selects complaint, attaches resolution proof photo -> **city_official.js** calls PATCH /api/complaints/{id} -> **api_client.py** updates status in DB -> SMS alert dispatched to resident.

Flow D: AI MegaPlan Generation

Official clicks 'Generate AI MegaPlan' -> **government.js** calls POST /api/megaplan -> **server.py** gathers complaints & BRICS node context -> **megaplan_engine.py** prompts Gemini AI -> returns structured multi-sector infrastructure plan.

Flow E: Cross-BRICS Collaboration

Central official files joint proposal -> **government.js** calls POST /api/brics/proposals -> **brics_proposals_engine.py** enriches via AI -> writes to *brics_incoming_requests.json* -> target nation official reviews on **central_official.html**.

Flow F: User Authentication & Role Access

User enters credentials on **login.html** -> **login.js** calls POST /api/auth/login or /api/auth/register -> **server.py** authenticates user -> returns JWT token & role -> redirects to designated portal.

4. Comprehensive Folder & File Directory Guide

Detailed specification of all system directories, their contents, purpose, and inter-dependencies:

Folder: *backend/* — *Python Server & Business Logic Core*

Description: Contains all server-side python execution logic, API endpoint controllers, database CRUD interfaces, and microservices.

• <code>server.py</code>	Main FastAPI/Starlette web server defining all API routes, authentication, file uploads, static mounts, and page serving.
• <code>api_client.py</code>	Database engine managing SQLite queries (CRUD) for complaints, users, SLA metrics, and engineer assignments.
• <code>gemini_helper.py</code>	Centralized wrapper for Google Gemini AI API providing severity triage, auto-summarization, and plan generation.
• <code>brics_context.py</code>	Geopolitical metadata store for 11 BRICS+ nations (capitals, currencies, infrastructure sectors).
• <code>brics_proposals_engine.py</code>	Handles inter-city CapEx proposals and cross-border BRICS joint venture requests.
• <code>demands_engine.py</code>	Manages citizen community demand proposals, upvoting, and country filtering.
• <code>megaplan_engine.py</code>	Synthesizes multi-sector AI MegaPlans from localized complaint clusters.
• <code>messaging_gateway.py</code> & <code>sms_gateway.py</code>	Dispatches real-time SMS and WhatsApp notifications for complaint updates.

Folder: *static/* — *Frontend Assets & Client Logic*

Description: Serves all client-side assets rendered directly by user web browsers.

• <code>static/pages/</code>	HTML5 portal views for each role: <code>login.html</code> , <code>citizen.html</code> , <code>city_official.html</code> , <code>government.html</code> , <code>central_official.html</code> .
• <code>static/js/</code>	Role controllers: <code>app.js</code> (bootstrap & icons), <code>i18n.js</code> (GNMT translation), <code>citizen.js</code> , <code>city_official.js</code> , <code>government.js</code> , <code>central_official.js</code> , <code>problem_map_table.js</code> (GIS Leaflet map).
• <code>static/css/styles.css</code>	Single unified enterprise dark-mode glassmorphism design system.
• <code>static/uploads/</code>	Runtime storage directory for user-uploaded issue photos and official resolution proof photos.

Folder: *data/* — *Persistence & Runtime Data Store*

Description: Stores active production database and structured JSON state files.

• <code>civicpulse.db</code>	Primary SQLite database storing complaints, registered users, SLA logs, and engineer assignments.
• <code>demands.json</code>	JSON flat-file tracking community project proposals and voter logs.

• <code>city_proposals.json</code>	JSON store for municipal CapEx infrastructure proposals.
• <code>brics_incoming_requests.json</code>	JSON store for cross-border BRICS joint venture requests.

Folder: `public/` — *Web Root & Static Server Entry*

Description: Root-level static entry points for web servers and static hostings.

• <code>index.html</code>	Top-level landing page and role selection entry point.
• <code>404.html</code>	Custom BRICS-branded 404 error page.

Folder: `assets/` & `scripts/` — *Static Media & Automated Utility Scripts*

Description: Bundled sector graphics and automation scripts for DB seeding and testing.

• <code>assets/</code>	Category thumbnail graphics (<code>cat_roads.jpg</code> , <code>cat_water.jpg</code> , <code>cat_power.jpg</code> , etc.) used in Citizen sector grid.
• <code>scripts/</code>	Automation scripts: <code>test_verification.py</code> (unit tests), <code>inspect_db.py</code> (DB inspector).

5. Deployment & Production Pipeline

Cloud Deployment: The application is packaged via **Dockerfile** and auto-deployed to Render.

Container Execution Command: `uvicorn backend.server:app --host 0.0.0.0 --port 8080`

Live Production URL: <https://civicpulse-brics.onrender.com>